Homework 3: The Expectation Maximization (EM) algorithm

STAT 348, UChicago, Spring 2026

Your name here: Oleg Glotov

Hours spent: 13

(Please let us know how many hours in total you spent on this assignment so we can calibrate for future assignments. Your feedback is always welcome!)

Open in Colab

Instructions

The purpose of this assignment is to explore the ideas in weeks 2-3:

- mixture models
- probabilistic graphical models
- conditional conjugacy
- the EM algorithm

Assignment is due **Monday April 20, 11:59pm** on GradeScope.

Zero-inflated Poisson Mixture Model

The following generative process defines the model we will be working with:

$$\boldsymbol{\pi} \sim \mathrm{Dir}(\boldsymbol{\alpha}_0) \tag{1}$$

$$\omega_{k,d} \overset{\text{iid}}{\sim} \mathrm{Beta}(a_0, b_0) \tag{2}$$

$$\lambda_{k,d} \overset{\text{iid}}{\sim} \mathrm{Gamma}(e_0, f_0) \tag{3}$$

$$z_i \overset{\text{iid}}{\sim} \mathrm{Cat}(\boldsymbol{\pi}) \tag{4}$$

$$b_{i,d} \overset{\text{ind.}}{\sim} \mathrm{Bernoulli}(\omega_{z_i,d}) \tag{5}$$

$$x_{i,d} \overset{\text{ind.}}{\sim} \begin{cases} \delta_0 & \text{if } b_{i,d} = 0 \\ \mathrm{Pois}(\lambda_{z_i,d}) & \text{if } b_{i,d} = 1 \end{cases} \tag{6}$$

- Each observation is a D-dimensional count vector $\boldsymbol{x}_i = (x_{i,1}, \ldots, x_{i,D}) \in \mathbb{N}_0^D$.
- Each $x_{i,d}$ is associated with a latent Bernoulli variable $b_{i,d}$. When $b_{i,d} = 1$, $x_{i,d}$ is sampled from a Poisson. When $b_{i,d} = 0$, $x_{i,d} = 0$ deterministically.
- Each observation is assigned to one of K mixture components $z_i \in \{1, \ldots, K\}$.
- We will often use the following transformation of z_i

$$\zeta_{ik} \triangleq 1(z_i = k), \;\; \boldsymbol{\zeta}_i = (\zeta_{i1}, \ldots, \zeta_{iK})$$

- An equivalent way to sample the cluster assignment is:

$$\boldsymbol{\zeta}_i \sim \text{Multinomial}(1, \boldsymbol{\pi})$$

- Each mixture component k is associated with two D-dimensional parameter vectors, $\boldsymbol{\omega}_k$ and $\boldsymbol{\lambda}_k$.
- The vector $\boldsymbol{\omega}_k = (\omega_{k,1}, \ldots, \omega_{k,D})$ gives the probabilities for the zero-inflated part of the likelihood where each $\omega_{k,d} \in (0, 1)$ is Beta distributed.
- The vector $\boldsymbol{\lambda}_k = (\lambda_{k,1}, \ldots, \lambda_{k,D})$ gives the rates for the Poisson part of the likelihood where each $\lambda_{k,d} > 0$ is Gamma distributed.
- The weights $\boldsymbol{\pi} \in \Delta_K$ give the prior probability of each component and are Dirichlet distributed.
- We will refer to the full set of model parameters as $\Theta = \{\Lambda, \Omega, \boldsymbol{\pi}\}$.
- We will refer to the full set of hyperparameters as $\boldsymbol{\eta}_0 = \{a_0, b_0, e_0, f_0, \boldsymbol{\alpha}_0\}$.

The **likelihood** of the model can be expressed as:

$$P(X \mid B, Z, \Theta, \boldsymbol{\eta}_0) = \prod_{i=1}^{n} \prod_{k=1}^{K} \prod_{d=1}^{D} \text{Pois}(x_{i,d}; \lambda_{k,d})^{\zeta_{i,k}\, b_{i,d}} \delta_0(x_{i,d})^{\zeta_{i,k}(1-b_{i,d})} \tag{7}$$

The **"complete data likelihood"** of the model is then $p(X, B, Z \mid \Theta, \boldsymbol{\eta}_0) = p(X \mid B, Z, \Theta, \boldsymbol{\eta}_0)\, p(B, Z \mid \Theta, \boldsymbol{\eta}_0)$ where

$$p(B, Z \mid \Theta, \boldsymbol{\eta}_0) = \prod_{i=1}^{n} \prod_{k=1}^{K} \left[p(z_i = k \mid \boldsymbol{\pi}) \prod_{d=1}^{D} p(b_{i,d} \mid \omega_{k,d}) \right]^{\zeta_{i,k}} \tag{8}$$

$$= \prod_{i=1}^{n} \prod_{k=1}^{K} \pi_k^{\zeta_{i,k}} \prod_{d=1}^{D} \omega_{k,d}^{\zeta_{i,k} b_{i,d}} (1 - \omega_{k,d})^{\zeta_{i,k}(1-b_{i,d})} \tag{9}$$

The **prior** over parameters $\Lambda, \Omega, \boldsymbol{\pi}$ is:

$$p(\Theta \mid \boldsymbol{\eta}_0) = p(\boldsymbol{\pi} \mid \boldsymbol{\alpha}) \prod_{k=1}^{K} \prod_{d=1}^{D} p(\omega_{k,d} \mid a_0, b_0)\, p(\lambda_{k,d} \mid e_0, f_0) \tag{10}$$

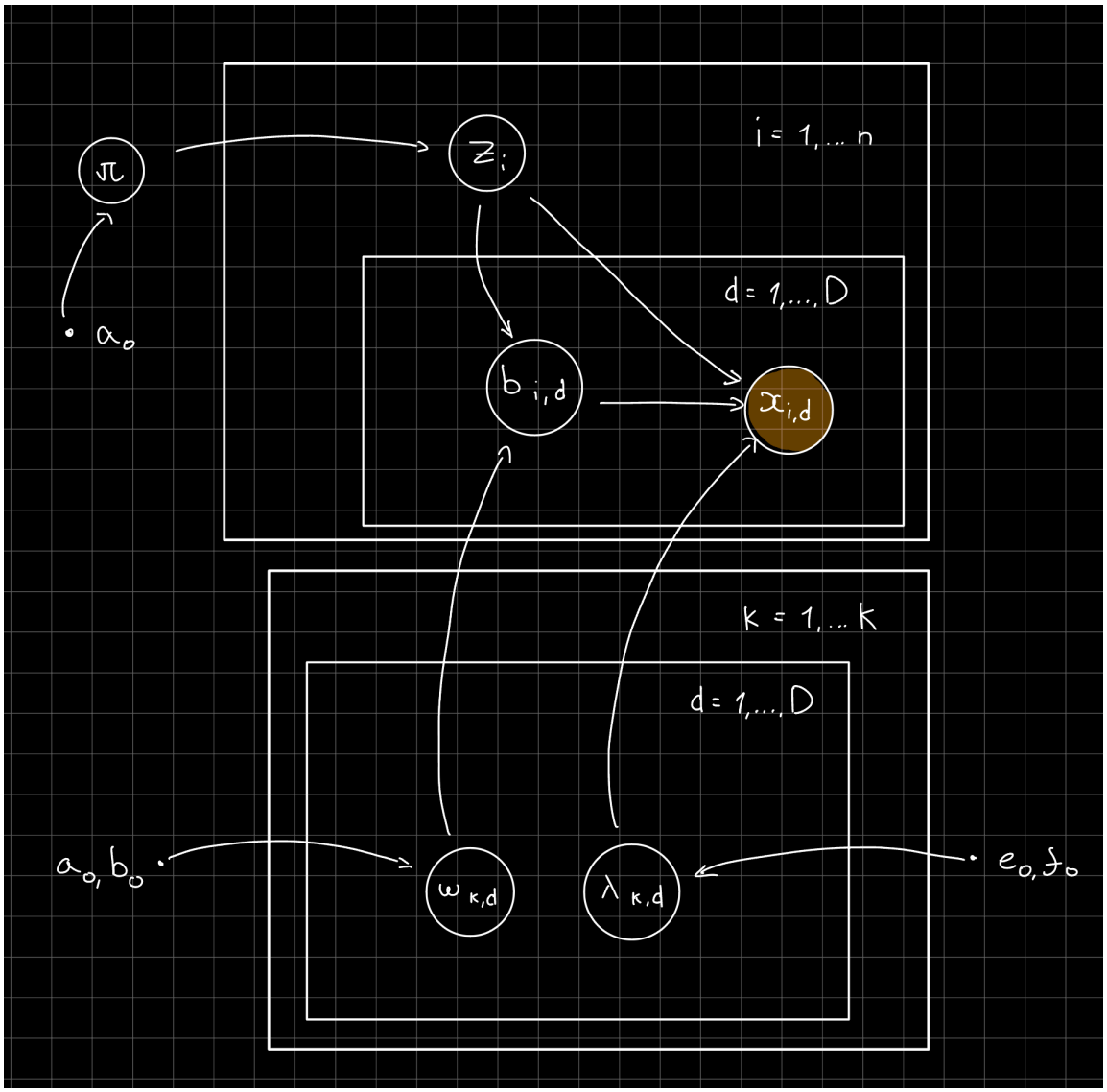
$$= \mathrm{Dir}(\boldsymbol{\pi};\, \boldsymbol{\alpha}) \prod_{k=1}^{K} \prod_{d=1}^{D} \mathrm{Beta}(\omega_{k,d}; a_0, b_0)\, \mathrm{Gamma}(\lambda_{k,d}; e_0, f_0) \tag{11}$$

Problem 0: Draw the graphical model [Visual].

Using either sofware (e.g., TikZ, Keynote) or **very neat handwriting**, create a probabilistic graphical model that describes the generative process above.

Your PGM should:

- Use plate notation to denote repeated sampling.
- Use shaded circular nodes to denote observed variables.
- Use un-shaded circular nodes to denote latent variables.
- Use square or bullet-nodes to denote hyperparameters.



Problem 1: Derive the complete conditionals [Math]

1a): Derive the complete conditional distribution $p(\boldsymbol{\pi} \mid -) \equiv p(\boldsymbol{\pi} \mid X, B, Z, \Theta_{\{\setminus \boldsymbol{\pi}\}}, \boldsymbol{\eta}_0)$.

π depends on its parent α_0 and its children $\{z_i\}_{i=1}^n$.

The complete conditional is proportional to the prior times the likelihood:

$$p(\pi \mid -) \propto p(\pi) \prod_{i=1}^{n} p(z_i \mid \pi)$$

The prior is Dirichlet:

$$p(\pi) \propto \prod_{k=1}^{K} \pi_k^{\alpha_{0,k}-1}$$

Each z_i is Categorical:

$$p(z_i \mid \pi) = \prod_{k=1}^{K} \pi_k^{\zeta_{i,k}}$$

Taking the product over i:

$$\prod_{i=1}^{n} p(z_i \mid \pi) = \prod_{k=1}^{K} \pi_k^{\sum_i \zeta_{i,k}}$$

Multiplying prior and likelihood:

$$p(\pi \mid -) \propto \prod_{k=1}^{K} \pi_k^{\alpha_{0,k}+\sum_i \zeta_{i,k}-1}$$

This is the kernel of a Dirichlet distribution. So:

$$p(\boldsymbol{\pi} \mid -) = \mathrm{Dir}\left(\alpha_{0,1} + \sum_i \zeta_{i,1},\ \ldots,\ \alpha_{0,K} + \sum_i \zeta_{i,K}\right)$$

1b): Derive the complete conditional distribution $p(\omega_{k,d} \mid -)$.

$\omega_{k,d}$ depends on its parents (a_0, b_0) and its children $\{b_{i,d}\}$ with co-parents $\{z_i\}$.

The complete conditional is:

$$p(\omega_{k,d} \mid -) \propto p(\omega_{k,d}) \prod_{i=1}^{n} p(b_{i,d} \mid \omega_{z_i,d})$$

Only terms where $z_i = k$ actually involve $\omega_{k,d}$ (the rest are constants). Using the ζ indicator:

$$\prod_{i=1}^{n} p(b_{i,d} \mid \omega_{z_i,d}) \propto \prod_{i=1}^{n} \left[\omega_{k,d}^{b_{i,d}} (1-\omega_{k,d})^{1-b_{i,d}}\right]^{\zeta_{i,k}}$$

Combining exponents:

$$= \omega_{k,d}^{\sum_i \zeta_{i,k} b_{i,d}} (1-\omega_{k,d})^{\sum_i \zeta_{i,k}(1-b_{i,d})}$$

The prior is Beta:

$$p(\omega_{k,d}) \propto \omega_{k,d}^{a_0-1} (1-\omega_{k,d})^{b_0-1}$$

Multiplying:

$$p(\omega_{k,d} \mid -) \propto \omega_{k,d}^{a_0+\sum_i \zeta_{i,k} b_{i,d}-1}(1-\omega_{k,d})^{b_0+\sum_i \zeta_{i,k}(1-b_{i,d})-1}$$

This is the kernel of a Beta dist. So:

$$p(\omega_{k,d} \mid -) = \text{Beta}\left(a_0 + \sum_i \zeta_{i,k} b_{i,d},\ b_0 + \sum_i \zeta_{i,k}(1-b_{i,d})\right)$$

1c): Derive the complete conditional distribution $p(\lambda_{k,d} \mid -)$.

$\lambda_{k,d}$ depends on its parents (e_0, f_0) and its children $\{x_{i,d}\}$ with co-parents $\{z_i, b_{i,d}\}$.

The complete conditional is:

$$p(\lambda_{k,d} \mid -) \propto p(\lambda_{k,d}) \prod_{i=1}^{n} p(x_{i,d} \mid b_{i,d}, \lambda_{z_i,d})^{\zeta_{i,k}}$$

When $b_{i,d} = 0$, the likelihood is $\delta_0(x_{i,d})$, which does not depend on λ. So only terms with $b_{i,d} = 1$ and $\zeta_{i,k} = 1$ contribute:

$$\propto p(\lambda_{k,d}) \prod_{i=1}^{n} \text{Pois}(x_{i,d}; \lambda_{k,d})^{\zeta_{i,k} b_{i,d}}$$

The Poisson pmf has kernel (in λ):

$$\text{Pois}(x; \lambda) \propto \lambda^x e^{-\lambda}$$

Taking the product:

$$\prod_{i=1}^{n} \left[\lambda_{k,d}^{x_{i,d}} e^{-\lambda_{k,d}}\right]^{\zeta_{i,k} b_{i,d}} = \lambda_{k,d}^{\sum_i \zeta_{i,k} b_{i,d} x_{i,d}} e^{-\lambda_{k,d} \sum_i \zeta_{i,k} b_{i,d}}$$

The prior is Gamma:

$$p(\lambda_{k,d}) \propto \lambda_{k,d}^{e_0-1} e^{-f_0 \lambda_{k,d}}$$

Multiplying prior and likelihood:

$$p(\lambda_{k,d} \mid -) \propto \lambda_{k,d}^{e_0+\sum_i \zeta_{i,k} b_{i,d} x_{i,d}-1} e^{-\lambda_{k,d}(f_0+\sum_i \zeta_{i,k} b_{i,d})}$$

This is the kernel of a Gamma dist. So:

$$p(\lambda_{k,d} \mid -) = \text{Gamma}\left(e_0 + \sum_i \zeta_{i,k} b_{i,d} x_{i,d},\ f_0 + \sum_i \zeta_{i,k} b_{i,d}\right)$$

1d): Derive the complete conditional distribution $p(b_{i,d} = 1 \mid z_i = k, -)$.

$b_{i,d}$ depends on its parent $\omega_{k,d}$ (via $z_i = k$), its child $x_{i,d}$, and co-parent $\lambda_{k,d}$.

The complete conditional is proportional to the prior times the likelihood:

$$p(b_{i,d} \mid z_i = k, -) \propto p(b_{i,d} \mid \omega_{k,d}) \cdot p(x_{i,d} \mid b_{i,d}, \lambda_{k,d})$$

Since $b_{i,d}$ is binary, I show the two cases.

For $b_{i,d} = 1$:

$$p(b_{i,d} = 1 \mid z_i = k, -) \propto \omega_{k,d} \cdot \mathrm{Pois}(x_{i,d}; \lambda_{k,d})$$

or

$$p(b_{i,d} = 1 \mid z_i = k, -) = \frac{\omega_{k,d}\, e^{-\lambda_{k,d}}}{\omega_{k,d}\, e^{-\lambda_{k,d}} + (1 - \omega_{k,d})}$$

For $b_{i,d} = 0$: the likelihood is the delta at zero, which is 1 if $x_{i,d} = 0$ and 0 otherwise:

$$p(b_{i,d} = 0 \mid z_i = k, -) \propto (1 - \omega_{k,d}) \cdot 1(x_{i,d} = 0)$$

We can normalize here, so final becomes:

$$p(b_{i,d} = 1 \mid z_i = k, -) = \begin{cases} 1 & \text{if } x_{i,d} > 0 \\ \dfrac{\omega_{k,d}\, e^{-\lambda_{k,d}}}{\omega_{k,d}\, e^{-\lambda_{k,d}} + (1 - \omega_{k,d})} & \text{if } x_{i,d} = 0 \end{cases}$$

1e): Derive the complete conditional distribution $p(z_i = k \mid -)$.

z_i depends on its parent π and its children $\{b_{i,d}, x_{i,d}\}_d$ with co-parents $\{\omega_{k,d}, \lambda_{k,d}\}$.

The complete conditional is:

$$p(z_i = k \mid -) \propto p(z_i = k \mid \pi) \prod_{d=1}^{D} p(b_{i,d} \mid \omega_{z_i,d})\, p(x_{i,d} \mid b_{i,d}, \lambda_{z_i,d})$$

Setting $z_i = k$:

$$\propto \pi_k \prod_{d=1}^{D} \omega_{k,d}^{b_{i,d}} (1 - \omega_{k,d})^{1 - b_{i,d}} \cdot \mathrm{Pois}(x_{i,d}; \lambda_{k,d})^{b_{i,d}}$$

(The $\delta_0(x_{i,d})^{1-b_{i,d}}$ terms don't depend on k, so they drop out.)

Normalizing over k:

$$p(z_i = k \mid -) = \frac{\pi_k \prod_d \omega_{k,d}^{b_{i,d}} (1-\omega_{k,d})^{1-b_{i,d}} \mathrm{Pois}(x_{i,d}; \lambda_{k,d})^{b_{i,d}}}{\sum_{j=1}^K \pi_j \prod_d \omega_{j,d}^{b_{i,d}} (1-\omega_{j,d})^{1-b_{i,d}} \mathrm{Pois}(x_{i,d}; \lambda_{j,d})^{b_{i,d}}}$$

1f): Derive the "incomplete" conditional distribution $p(z_i = k \mid -\backslash_{\boldsymbol{b}_i})$.

The marginal likelihood $p(x_{i,d} \mid z_i = k, \omega, \lambda)$ with $b_{i,d}$ marginalized out:

$$p(x_{i,d} \mid z_i = k, \omega, \lambda) = \sum_{b \in \{0,1\}} p(b \mid \omega_{k,d})\, p(x_{i,d} \mid b, \lambda_{k,d})$$

Expanding:

$$= (1-\omega_{k,d})\, \delta_0(x_{i,d}) + \omega_{k,d}\, \mathrm{Pois}(x_{i,d}; \lambda_{k,d})$$

This is the zero-inflated Poisson marginal likelihood.

Now similar to 1e but using this marginal likelihood in place of $p(b_{i,d} \mid \omega) p(x_{i,d} \mid b, \lambda)$:

$$p(z_i = k \mid -\backslash b_i) \propto \pi_k \prod_{d=1}^D \left[(1-\omega_{k,d})\, \delta_0(x_{i,d}) + \omega_{k,d}\, \mathrm{Pois}(x_{i,d}; \lambda_{k,d})\right]$$

Normalizing over k:

$$p(z_i = k \mid -\backslash_{\boldsymbol{b}_i}) = \frac{\pi_k \prod_d \left[(1-\omega_{k,d})\, \delta_0(x_{i,d}) + \omega_{k,d}\, \mathrm{Pois}(x_{i,d}; \lambda_{k,d})\right]}{\sum_{j=1}^K \pi_j \prod_d \left[(1-\omega_{j,d})\, \delta_0(x_{i,d}) + \omega_{j,d}\, \mathrm{Pois}(x_{i,d}; \lambda_{j,d})\right]}$$

Problem 2: Derive the EM algorithm

In this part, we will implement the EM algorithm to do **MAP estimation** for the parameters:

$$\widehat{\Theta}^{\mathrm{MAP}} = \arg\max_{\boldsymbol{\Theta}} p(\Theta \mid X, \boldsymbol{\eta}_0) \tag{12}$$

As a reminder, the EM algorithm finds a local mode of the posterior by performing coordinate ascent on the **evidence lower bound (ELBO)**:

$$B(q, \Theta) = \mathbb{E}_{q(B,Z)} \left[\log \frac{p(X, B, Z, \Theta \mid \boldsymbol{\eta}_0)}{q(B, Z)}\right] \tag{13}$$

More specifically, EM iterates maximizing the ELBO with respect to q (E-step) and then with respect to Θ (M-step). The optimal updates for both steps can be written as:

$$\text{(E-step)} \qquad q = \arg\max_q B(q, \Theta^{\text{old}}) = p(B, Z \mid \Theta^{\text{old}}, \boldsymbol{\eta}_0) \qquad (1$$

$$\text{(M-step)} \qquad \Theta^{\text{new}} = \arg\max_\Theta B(q, \Theta) = \arg\max_{\boldsymbol{\Theta}} \mathbb{E}_q \left[\log p(X, B, Z, \Theta \mid \boldsymbol{\eta}_0)\right] \quad (1$$

We will help you by telling you that for the E-step, it will suffice to compute the following posterior expectations:

$$q(z_i = k) \equiv \mathbb{E}_q[\zeta_{i,k}] \qquad \text{for all } i, k \qquad (16)$$

$$q(b_{i,d} = 1 \mid z_i = k) \equiv \mathbb{E}_q[b_{i,d} \mid z_i = k] \qquad \text{for all } i, d, k \qquad (17)$$

We will further help you telling you that the M-step in this model can be equivalently written as performing the following separate steps:

$$\boldsymbol{\pi} \leftarrow \arg\max_{\boldsymbol{\pi}} \mathbb{E}_q \left[\log P(\boldsymbol{\pi} \mid -)\right] \qquad (18)$$

$$\omega_{k,d} \leftarrow \arg\max_{\omega_{k,d}} \mathbb{E}_q \left[\log P(\omega_{k,d} \mid -)\right] \qquad \text{for all } k, d \qquad (19)$$

$$\lambda_{k,d} \leftarrow \arg\max_{\lambda_{k,d}} \mathbb{E}_q \left[\log P(\lambda_{k,d} \mid -)\right] \qquad \text{for all } k, d \qquad (20)$$

2a) Derive the E-step.

Derive and provide forms for the posterior expectations $\mathbb{E}_q[\zeta_{i,k}]$ and $\mathbb{E}_q[b_{i,d} \mid z_i = k]$ described above.

We need two quantities:

$$q(z_i = k) \qquad \text{and} \qquad q(b_{i,d} = 1 \mid z_i = k)$$

For $q(z_i = k)$: This is $p(z_i = k \mid X, \Theta)$ marginalizing over b_i. From 1f:

$$q(z_i = k) = \frac{\pi_k \prod_d \left[(1 - \omega_{k,d})\delta_0(x_{i,d}) + \omega_{k,d}\text{Pois}(x_{i,d}; \lambda_{k,d})\right]}{\sum_{j=1}^K \pi_j \prod_d \left[(1 - \omega_{j,d})\delta_0(x_{i,d}) + \omega_{j,d}\text{Pois}(x_{i,d}; \lambda_{j,d})\right]}$$

For $q(b_{i,d} = 1 \mid z_i = k)$: This is $p(b_{i,d} = 1 \mid z_i = k, x_{i,d}, \Theta)$. From 1d:

$$q(b_{i,d} = 1 \mid z_i = k) = \begin{cases} 1 & \text{if } x_{i,d} > 0 \\ \dfrac{\omega_{k,d} e^{-\lambda_{k,d}}}{\omega_{k,d} e^{-\lambda_{k,d}} + (1 - \omega_{k,d})} & \text{if } x_{i,d} = 0 \end{cases}$$

2b) Derive the M-step.

Derive and provide forms for the M-step updates to the parameters.

Let $r_{i,k} = q(z_i = k)$ and $s_{i,d,k} = q(b_{i,d} = 1 \mid z_i = k)$. Since q factors, we have:

$$\mathbb{E}_q[\zeta_{i,k}] = r_{i,k}, \qquad \mathbb{E}_q[\zeta_{i,k} b_{i,d}] = r_{i,k} s_{i,d,k}$$

Here, using closed-form modes for Dirichlet, Beta, Gamma:

1. Mode of $\mathrm{Dir}(\alpha_1, \ldots, \alpha_K)$: $\pi_k = (\alpha_k - 1)/(\sum_j \alpha_j - K)$
2. Mode of $\mathrm{Beta}(a, b)$: $(a-1)/(a+b-2)$
3. Mode of $\mathrm{Gamma}(e, f)$: $(e-1)/f$

Therefore to update for π: From 1a, the complete conditional is $\mathrm{Dir}(\alpha_{0,k} + \sum_i \zeta_{i,k})$. Taking the expectation of the log and then the mode:

$$\pi_k = \frac{\alpha_{0,k} + \sum_i r_{i,k} - 1}{\sum_{j=1}^K \alpha_{0,j} + n - K}$$

From 1b, the complete conditional is $\mathrm{Beta}(a_0 + \sum_i \zeta_{i,k} b_{i,d},\ b_0 + \sum_i \zeta_{i,k}(1 - b_{i,d}))$. Taking expectations and then the Beta mode:

$$\omega_{k,d} = \frac{a_0 + \sum_i r_{i,k} s_{i,d,k} - 1}{a_0 + b_0 + \sum_i r_{i,k} - 2}$$

From 1c, the complete conditional is $\mathrm{Gamma}(e_0 + \sum_i \zeta_{i,k} b_{i,d} x_{i,d},\ f_0 + \sum_i \zeta_{i,k} b_{i,d})$. Taking expectations and then the Gamma mode:

$$\lambda_{k,d} = \frac{e_0 + \sum_i r_{i,k} s_{i,d,k} x_{i,d} - 1}{f_0 + \sum_i r_{i,k} s_{i,d,k}}$$

2c) Derive the ELBO.

Derive an analytic form for the ELBO. Because this can be tricky, we encourage you to break down the derivation in to four separate pieces:

$$B(q, \Theta) = \mathbb{E}_{q(B,Z)} \left[\log \frac{p(X, B, Z, \Theta \mid \boldsymbol{\eta}_0)}{q(B, Z)} \right] \tag{21}$$

$$= \mathbb{E}_q \Big[\log p(X \mid B, Z, \Theta, \boldsymbol{\eta}_0) \Big] \tag{22}$$

$$+ \mathbb{E}_q \Big[\log p(B, Z \mid \Theta, \boldsymbol{\eta}_0) \Big] \tag{23}$$

$$- \mathbb{E}_q \Big[\log q(B, Z) \Big] \tag{24}$$

$$+ \log p(\Theta \mid \boldsymbol{\eta}_0) \tag{25}$$

Using the same notation $r_{i,k} = q(z_i = k)$ and $s_{i,d,k} = q(b_{i,d} = 1 \mid z_i = k)$.

$\mathbb{E}_q[\log p(X \mid B, Z, \Theta)]$

From the likelihood:

$$\log p(X \mid B, Z, \Theta) = \sum_{i,k,d} \zeta_{i,k} b_{i,d} \log \mathrm{Pois}(x_{i,d}; \lambda_{k,d}) + \zeta_{i,k}(1 - b_{i,d}) \log \delta_0(x_{i,d})$$

The $\log \delta_0(x_{i,d})$ term is 0 whenever $x_{i,d} = 0$ so it drops. Taking the expectation:

$$\mathbb{E}_q[\log p(X \mid B, Z, \Theta)] = \sum_{i,k,d} r_{i,k} s_{i,d,k} \log \mathrm{Pois}(x_{i,d}; \lambda_{k,d})$$

2

$\mathbb{E}_q[\log p(B, Z \mid \Theta)]$

$$\log p(B, Z \mid \Theta) = \sum_{i,k} \zeta_{i,k} \log \pi_k + \sum_{i,k,d} \zeta_{i,k} \left[b_{i,d} \log \omega_{k,d} + (1 - b_{i,d}) \log(1 - \omega_{k,d}) \right]$$

Taking expectations:

$$\mathbb{E}_q[\log p(B, Z \mid \Theta)] = \sum_{i,k} r_{i,k} \log \pi_k + \sum_{i,k,d} r_{i,k} \left[s_{i,d,k} \log \omega_{k,d} + (1 - s_{i,d,k}) \log(1 - \omega_{k,d}) \right.$$

3

$-\mathbb{E}_q[\log q(B, Z)]$

Since $q(B, Z) = q(Z) \prod_i q(B_i \mid z_i)$:

$$-\mathbb{E}_q[\log q(Z)] = -\sum_{i,k} r_{i,k} \log r_{i,k}$$

$$-\mathbb{E}_q[\log q(B \mid Z)] = -\sum_{i,k,d} r_{i,k} \left[s_{i,d,k} \log s_{i,d,k} + (1 - s_{i,d,k}) \log(1 - s_{i,d,k}) \right]$$

4

$\log p(\Theta \mid \eta_0)$

Given.

$$\log p(\Theta \mid \eta_0) = \log \mathrm{Dir}(\pi; \alpha_0) + \sum_{k,d} \log \mathrm{Beta}(\omega_{k,d}; a_0, b_0) + \sum_{k,d} \log \mathrm{Gamma}(\lambda_{k,d}; e_0, f$$

ELBO

Summing the four parts:

$$
\begin{aligned}
B(q, \Theta) = &\sum_{i,k,d} r_{i,k} s_{i,d,k} \log \mathrm{Pois}(x_{i,d}; \lambda_{k,d}) \\
&+ \sum_{i,k} r_{i,k} \log \pi_k + \sum_{i,k,d} r_{i,k} \left[s_{i,d,k} \log \omega_{k,d} + (1 - s_{i,d,k}) \log(1 - \omega_{k,d}) \right] \\
&- \sum_{i,k} r_{i,k} \log r_{i,k} - \sum_{i,k,d} r_{i,k} \left[s_{i,d,k} \log s_{i,d,k} + (1 - s_{i,d,k}) \log(1 - s_{i,d,k}) \right] \\
&+ \log p(\Theta \mid \eta_0)
\end{aligned}
$$

Question 3: Implement EM [Code]

In this question you will implement the full EM algorithm using PyTorch for MAP estimation in the zero-inflated Poisson mixture model.

We have provided a skeleton structure for the code you should implement below. The function `em` below will run EM on the inputs. This function is fully implemented, and you do not need to edit it. However, this function calls the following functions:

- `initialize_em` : initializes the parameters and posterior expectations
- `e_step` : runs the E-step and returns an updated version of the posterior expectations
- `m_step` : runs the M-step and returns an updated version of the parameters
- `elbo` : calculates the evidence lower bound (ELBO)

For your convenience, we have also fully implemented `initialize_em` . Your job is therefore to implement the other three functions (i.e., `e_step` , `m_step` , `elbo`) to get a fully working implementation of `em` .

By default, the functions you will implement take the following four dictionaries as arguments:

- `data` : contains X
- `belief_state` : contains the posterior expectations $\mathbb{E}_q[\zeta_{i,k}]$ and $\mathbb{E}_q[b_{i,d} \mid z_i = k]$
- `params_state` : contains the parameters Θ
- `hyperparams` : contains the hyperparameters $\boldsymbol{\eta}_0$

Each function may or may not require all of the information in all dictionaries. You can also see the names assigned to the different variables in `initialize_em` , and we encourage you not to change those.

In [70]:
```
import torch
from tqdm.notebook import tqdm

def initialize_em(data, hyperparams, seed=617):
    """Initialize the parameters and beliefs for the EM algorithm for the Ze
```

```
    Args:
        data (dict): dictionary of observed variables
        hyperparams (dict): dictionary of hyperparameters

    Returns:
        params_state (dict): dictionary of parameters
        belief_state (dict): dictionary of expectations of latent variables
    """
    # set the random seed for torch
    torch.manual_seed(seed)

    # get hyperparameters and sizes
    n, D = data['X_ND'].shape
    K = hyperparams['alpha_K'].shape[0]
    a0, b0, e0, f0, alpha_K = [hyperparams[key] for key in ['a0', 'b0', 'e0'

    # initialize the parameters with a draw from the prior
    params_state = {}
    params_state['pi_K'] = torch.distributions.Dirichlet(alpha_K).sample()
    params_state['omega_KD'] = torch.distributions.Beta(a0, b0).sample((K, D
    params_state['lambd_KD'] = torch.distributions.Gamma(concentration=e0, r

    # initialize the "beliefs" (posterior expectations) uniformly
    # (this is not important because the e-step is run first)
    belief_state = {}
    # this will store all the q(zi=k) = E[zeta_ik] expectations
    belief_state['Ezeta_NK'] = torch.ones((n, K)) / K
    # this will store all the q(bid=1 | zi=k) = E[bid | zi=k] expectations
    belief_state['Eb_NKD'] = torch.ones((n, K, D)) * 0.75
    return params_state, belief_state

def e_step(data, belief_state, params_state, hyperparams):
    """Perform the E-step of the EM algorithm for the Zero-Inflated Poisson

    Args:
        data (dict): dictionary of observed variables
        belief_state (dict): dictionary of expectations of latent variables
        params_state (dict): dictionary of parameters
        hyperparams (dict): dictionary of hyperparameters

    Returns:
        belief_state (dict): updated dictionary of expectations of latent va
    """
    # Your code here
    return belief_state

def m_step(data, belief_state, params_state, hyperparams):
    """Perform the M-step of the EM algorithm for the Zero-Inflated Poisson

    Args:
        data (dict): dictionary of observed variables
        belief_state (dict): dictionary of expectations of latent variables
        params_state (dict): dictionary of parameters
        hyperparams (dict): dictionary of hyperparameters
```

```
    Returns:
        params_state (dict): updated dictionary of parameters
    """
    # Your code
    return params_state

def elbo(data, params_state, belief_state, hyperparams):
    """
    Compute the Evidence Lower Bound (ELBO) for the Zero-Inflated Poisson Mi

    Args:
        data (dict): Dictionary containing observed data.
        params_state (dict): Dictionary containing model parameters.
        belief_state (dict): Dictionary containing expectations of latent va
        hyperparams (dict): Dictionary containing hyperparameters.

    Returns:
        torch.Tensor: The computed ELBO value.
    """
    # Your code here

def em(data, hyperparams, max_iter=100, tol=1e-7, seed=617, verbose=True):
    """Run the EM algorithm for the Zero-Inflated Poisson Mixture Model

    Args:
        data (dict): dictionary of observed variables
        hyperparams (dict): dictionary of hyperparameters
        max_iter (int): maximum number of iterations
        tol (float): tolerance for convergence
    """
    params_state, belief_state = initialize_em(data, hyperparams, seed=seed)
    elbo_vals = [elbo(data, params_state, belief_state, hyperparams)]

    # only display a progress bar if verbose=True
    iterator = tqdm(range(max_iter)) if verbose else range(max_iter)
    for _ in iterator:
        belief_state = e_step(data, belief_state, params_state, hyperparams)
        params_state = m_step(data, belief_state, params_state, hyperparams)
        elbo_vals.append(elbo(data, params_state, belief_state, hyperparams)

        curr_elbo_val, last_elbo_val = elbo_vals[-1], elbo_vals[-2]
        assert curr_elbo_val >= last_elbo_val

        # assess convergence (can also be done by looking at the param value
        if (curr_elbo_val - last_elbo_val).abs() < tol:
            break

    return params_state, belief_state, elbo_vals
```

3a) Implement the E-step.

- **Do not use for loops.** All calculations can and should be done via broadcasting and indexing.

- Make use of torch.logaddexp or torch.logsumexp to **compute (products of) probabilities in log-space** for numerically stable computation. (If your implementation is generating NaNs or INFs, it is often due to not properly doing things in log-space.)

- For your convenience, we have provided a **PyTorch implementation of the Poisson's log PMF** in the function `poisson_logpmf`. This function will **broadcast** (i.e., you can pass in tensors).

In [71]:
```
def poisson_logpmf(x, rate):
    """
    Compute the log probability mass function of the Poisson distribution.

    Args:
        x (torch.Tensor): The observed data.
        rate (torch.Tensor): The rate parameter (lambda) of the Poisson dist

    Returns:
        torch.Tensor: The log probability mass function of the Poisson distr
    """
    return x * torch.log(rate) - rate - torch.lgamma(x + 1)

def e_step(data, belief_state, params_state, hyperparams):
    """Perform the E-step of the EM algorithm for the Zero-Inflated Poisson

    Args:
        data (dict): dictionary of observed variables
        belief_state (dict): dictionary of expectations of latent variables
        params_state (dict): dictionary of parameters
        hyperparams (dict): dictionary of hyperparameters

    Returns:
        belief_state (dict): updated dictionary of expectations of latent va
    """
    X_ND = data['X_ND']
    pi_K = params_state['pi_K']
    omega_KD = params_state['omega_KD']
    lambd_KD = params_state['lambd_KD']

    n, D = X_ND.shape
    K = pi_K.shape[0]

    X_NKD = X_ND[:, None, :]
    omega_1KD = omega_KD[None, :, :]
    lambd_1KD = lambd_KD[None, :, :]


    # log Pois(x | lambda)
    log_pois_NKD = poisson_logpmf(X_NKD, lambd_1KD)

    # b=1
    log_on_NKD = torch.log(omega_1KD) + log_pois_NKD
```

```
    # b=0
    log_delta_ND = torch.where(X_ND == 0,
                               torch.tensor(0.0),
                               torch.tensor(float('-inf')))
    log_off_NKD = torch.log(1 - omega_1KD) + log_delta_ND[:, None, :]

    # E[b_{i,d} | z_i = k] = exp(log_on) / (exp(log_on) + exp(log_off))
    log_denom_NKD = torch.logaddexp(log_on_NKD, log_off_NKD)
    Eb_NKD = torch.exp(log_on_NKD - log_denom_NKD)

    log_ezeta_NK = torch.log(pi_K)[None, :] + log_denom_NKD.sum(dim=-1)

    # Normalize
    log_ezeta_NK = log_ezeta_NK - torch.logsumexp(log_ezeta_NK, dim=-1, keep
    Ezeta_NK = torch.exp(log_ezeta_NK)

    belief_state['Ezeta_NK'] = Ezeta_NK
    belief_state['Eb_NKD'] = Eb_NKD

    assert belief_state['Ezeta_NK'].shape == (n, K)
    assert belief_state['Eb_NKD'].shape == (n, K, D)
    return belief_state
```

In [72]:

```
# TESTING (gpt generated)

torch.manual_seed(0)
n, D, K = 20, 5, 3
X_ND_test = torch.poisson(torch.ones(n, D) * 2.0)
X_ND_test[X_ND_test < 2] = 0

data_test = {'X_ND': X_ND_test}
hypers_test = {
    'alpha_K': torch.ones(K),
    'a0': torch.tensor(2.0),
    'b0': torch.tensor(2.0),
    'e0': torch.tensor(2.0),
    'f0': torch.tensor(1.0),
}

params_state, belief_state = initialize_em(data_test, hypers_test)
belief_state = e_step(data_test, belief_state, params_state, hypers_test)

print("Ezeta_NK shape:", belief_state['Ezeta_NK'].shape, "(expected: (20, 3)
print("Eb_NKD shape:  ", belief_state['Eb_NKD'].shape, "(expected: (20, 3, 5

print("Ezeta row sums:", belief_state['Ezeta_NK'].sum(dim=-1)[:3], "(expecte
```

```
Ezeta_NK shape: torch.Size([20, 3]) (expected: (20, 3))
Eb_NKD shape:   torch.Size([20, 3, 5]) (expected: (20, 3, 5))
Ezeta row sums: tensor([1.0000, 1.0000, 1.0000]) (expected: all 1.0)
```

3b) Implement the M-step.

- **Do not use for loops.** All calculations can and should be done via broadcasting and indexing.

In [73]:
```
def m_step(data, belief_state, params_state, hyperparams):
    """Perform the M-step of the EM algorithm for the Zero-Inflated Poisson

    Args:
        data (dict): dictionary of observed variables
        belief_state (dict): dictionary of expectations of latent variables
        params_state (dict): dictionary of parameters
        hyperparams (dict): dictionary of hyperparameters

    Returns:
        params_state (dict): updated dictionary of parameters
    """

    X_ND = data['X_ND']
    Ezeta_NK = belief_state['Ezeta_NK']
    Eb_NKD = belief_state['Eb_NKD']

    alpha_K = hyperparams['alpha_K']
    a0 = hyperparams['a0']
    b0 = hyperparams['b0']
    e0 = hyperparams['e0']
    f0 = hyperparams['f0']

    n, K = Ezeta_NK.shape

    weighted_b_NKD = Ezeta_NK[:, :, None] * Eb_NKD

    sum_rs_KD = weighted_b_NKD.sum(dim=0)
    sum_ezeta_K = Ezeta_NK.sum(dim=0)

    # pi_k = (alpha_0_k + sum_i r_{i,k} - 1) / (sum_j alpha_0_j + n - K)
    pi_K = (alpha_K + sum_ezeta_K - 1) / (alpha_K.sum() + n - K)

    # omega_{k,d} = (a0 + sum_i r*s - 1) / (a0 + b0 + sum_i r - 2)
    omega_KD = (a0 + sum_rs_KD - 1) / (a0 + b0 + sum_ezeta_K[:, None] - 2)

    # lambda_{k,d} = (e0 + sum_i r*s*x - 1) / (f0 + sum_i r*s)
    sum_rsx_KD = (weighted_b_NKD * X_ND[:, None, :]).sum(dim=0)
    lambd_KD = (e0 + sum_rsx_KD - 1) / (f0 + sum_rs_KD)

    params_state['pi_K'] = pi_K
    params_state['omega_KD'] = omega_KD
    params_state['lambd_KD'] = lambd_KD

    return params_state
```

In [74]:
```
# TEST CELL

params_state = m_step(data_test, belief_state, params_state, hypers_test)

print("pi_K shape:    ", params_state['pi_K'].shape, "(expected: (3,))")
print("omega_KD shape:", params_state['omega_KD'].shape, "(expected: (3, 5))
```

```
print("lambd_KD shape:", params_state['lambd_KD'].shape, "(expected: (3, 5))

print("pi_K:          ", params_state['pi_K'])
print("pi_K sums to 1?", torch.allclose(params_state['pi_K'].sum(), torch.te

print("omega in (0,1)?", ((params_state['omega_KD'] > 0) & (params_state['om
print("lambda > 0?    ", (params_state['lambd_KD'] > 0).all().item())

print("No NaN/Inf?",
      torch.isfinite(params_state['pi_K']).all().item()
      and torch.isfinite(params_state['omega_KD']).all().item()
      and torch.isfinite(params_state['lambd_KD']).all().item())
```

```
pi_K shape:     torch.Size([3]) (expected: (3,))
omega_KD shape: torch.Size([3, 5]) (expected: (3, 5))
lambd_KD shape: torch.Size([3, 5]) (expected: (3, 5))
pi_K:           tensor([0.1291, 0.1904, 0.6805])
pi_K sums to 1? True
omega in (0,1)? True
lambda > 0?     True
No NaN/Inf? True
```

3c) Implement the ELBO.

- As in the derivation part, we encourage you to break down your implementation into the following four pieces:

$$B(q, \Theta) = \mathbb{E}_{q(B,Z)}\left[\log \frac{p(X, B, Z, \Theta \mid \boldsymbol{\eta}_0)}{q(B, Z)}\right]$$

$$= \mathbb{E}_q\Big[\log p(X \mid B, Z, \Theta, \boldsymbol{\eta}_0)\Big] + \mathbb{E}_q\Big[\log p(B, Z \mid \Theta, \boldsymbol{\eta}_0)\Big] - \mathbb{E}_q\Big[\log q$$

- For your convenience, **we have implemented the last piece** $\log p(\Theta \mid \boldsymbol{\eta}_0)$, which makes use of our PyTorch implementations of the log PDFs of Gamma, Beta, Dirichlet. Your job is to finish the implementation of the three other pieces.

- And again, **do not use for loops.**

In [75]:
```
def dirichlet_logpdf(x, alpha):
    """
    Compute the log probability density function of the Dirichlet distributi

    Args:
        x (torch.Tensor): The observed data. Should be a probability vector
        alpha (torch.Tensor): The concentration parameters of the Dirichlet

    Returns:
        torch.Tensor: The log probability density function of the Dirichlet
    """
    return torch.lgamma(alpha.sum()) - torch.lgamma(alpha).sum() + ((alpha -
```

```python
def gamma_logpdf(x, concentration, rate):
    """
    Compute the log probability density function of the Gamma distribution.

    Args:
        x (torch.Tensor): The observed data.
        concentration (torch.Tensor): The shape parameter of the Gamma distr
        rate (torch.Tensor): The rate parameter (1/scale) of the Gamma distr

    Returns:
        torch.Tensor: The log probability density function of the Gamma dist
    """
    return (concentration - 1) * torch.log(x) - x * rate - torch.lgamma(conc

def beta_logpdf(x, alpha, beta):
    """
    Compute the log probability density function of the Beta distribution.

    Args:
        x (torch.Tensor): The observed data. Should be in the range [0, 1].
        alpha (torch.Tensor): The alpha parameter of the Beta distribution.
        beta (torch.Tensor): The beta parameter of the Beta distribution.

    Returns:
        torch.Tensor: The log probability density function of the Beta distr
    """
    return torch.lgamma(alpha + beta) - torch.lgamma(alpha) - torch.lgamma(b

def elbo(data, params_state, belief_state, hyperparams):
    """
    Compute the Evidence Lower Bound (ELBO) for the Zero-Inflated Poisson Mi

    Args:
        data (dict): Dictionary containing observed data.
        params_state (dict): Dictionary containing model parameters.
        belief_state (dict): Dictionary containing expectations of latent va
        hyperparams (dict): Dictionary containing hyperparameters.

    Returns:
        torch.Tensor: The computed ELBO value.
    """
    a0, b0, e0, f0, alpha_K = [hyperparams[key] for key in ['a0', 'b0', 'e0'
    pi_K = params_state['pi_K']
    omega_KD = params_state['omega_KD']
    lambd_KD = params_state['lambd_KD']

    # log P(params)
    logptheta = dirichlet_logpdf(pi_K, alpha_K)
    logptheta += beta_logpdf(omega_KD, a0, b0).sum()
    logptheta += gamma_logpdf(lambd_KD, e0, rate=f0).sum()


    X_ND = data['X_ND']
    Ezeta_NK = belief_state['Ezeta_NK']
    Eb_NKD = belief_state['Eb_NKD']
```

```
    # Expected counts r_{i,k} * s_{i,d,k}
    weighted_b_NKD = Ezeta_NK[:, :, None] * Eb_NKD

    # Eq[ log p(x | b, z) ]
    log_pois_NKD = poisson_logpmf(X_ND[:, None, :], lambd_KD[None, :, :])
    Elogpx = (weighted_b_NKD * log_pois_NKD).sum()


    # Eq[ log p(b, z) ]
    # = sum_{i,k} r * log pi_k + sum_{i,k,d} r * [s*log omega + (1-s)*log(1-
    Elogpz = (Ezeta_NK * torch.log(pi_K)[None, :]).sum()
    log_omega_KD = torch.log(omega_KD)
    log_1m_omega_KD = torch.log(1 - omega_KD)
    Elogpb = (weighted_b_NKD * log_omega_KD[None, :, :] + (Ezeta_NK[:, :, No
    Elogpbz = Elogpz + Elogpb


    # Eq[ log q(b, z)]
    # = E[log q(Z)] + E[log q(B|Z)]
    zero = 1e-30 # =~ 0
    Elogqz = (Ezeta_NK * torch.log(Ezeta_NK + zero)).sum()
    Elogqb = (weighted_b_NKD * torch.log(Eb_NKD + zero)
              + (Ezeta_NK[:, :, None] - weighted_b_NKD) * torch.log(1 - Eb_N
    Elogqbz = Elogqz + Elogqb


    return Elogpx + Elogpbz - Elogqbz + logptheta
```

In [76]:

```
# TEST CELL

# Run e_step then m_step, then compute ELBO, then verify it goes up after an
belief_state = e_step(data_test, belief_state, params_state, hypers_test)
elbo_1 = elbo(data_test, params_state, belief_state, hypers_test)

params_state = m_step(data_test, belief_state, params_state, hypers_test)
elbo_2 = elbo(data_test, params_state, belief_state, hypers_test)

belief_state = e_step(data_test, belief_state, params_state, hypers_test)
elbo_3 = elbo(data_test, params_state, belief_state, hypers_test)

print(f"ELBO after e_step:  {elbo_1.item():.4f}")
print(f"ELBO after m_step:  {elbo_2.item():.4f}")
print(f"ELBO after e_step:  {elbo_3.item():.4f}")
print(f"Monotonically increasing? {elbo_1 <= elbo_2 <= elbo_3}")
print(f"ELBO finite? {torch.isfinite(elbo_1) and torch.isfinite(elbo_2) and
```

```
ELBO after e_step:  -177.7910
ELBO after m_step:  -177.1840
ELBO after e_step:  -176.8248
Monotonically increasing? True
ELBO finite? True
```

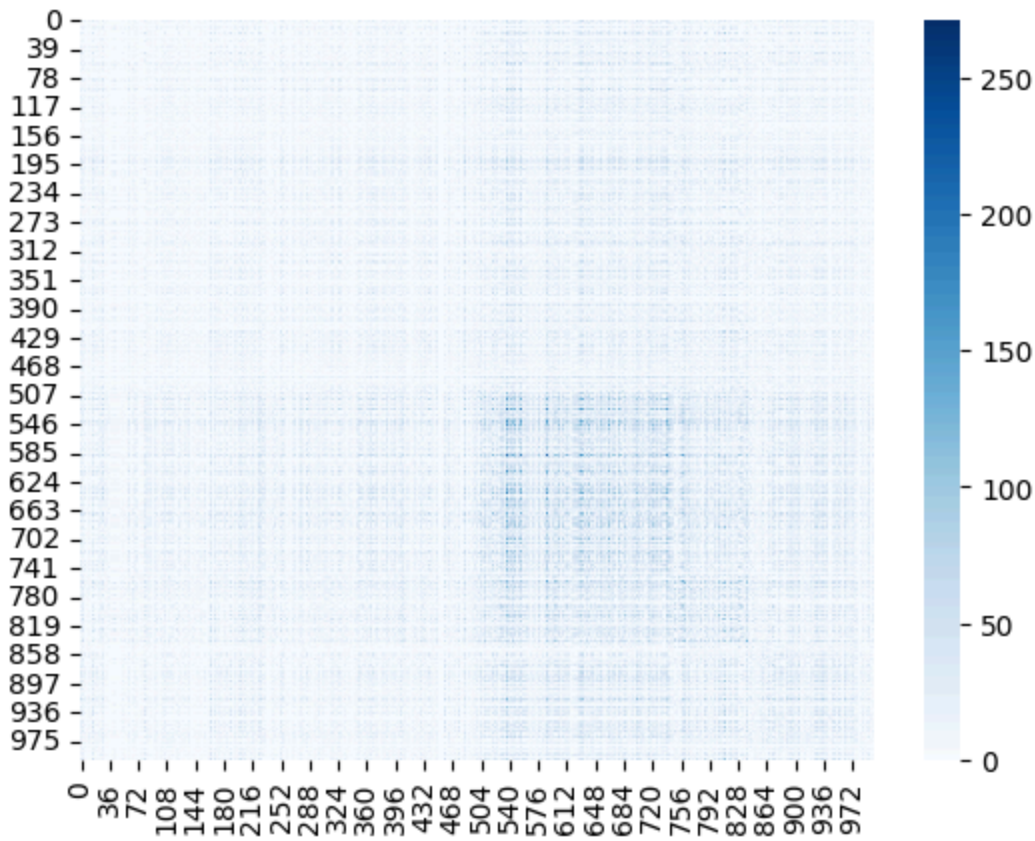
Question 4: Results on "real" data [Code, results]

In this question, you will run your EM algorithm on a data set with provided hyperparameters. First, load in the data and hyperparameters as dictionaries.

In [77]:
```
import pickle

with open('data.pkl', 'rb') as f:
    data = pickle.load(f)

with open('hyperparams.pkl', 'rb') as f:
    hyperparams = pickle.load(f)
```

The data is synthetic and has been generated by the model itself. We have re-ordered the rows of the data matrix X so that observations that were part of the same (true) cluster are adjacent. (In a realistic setting, we would not know the true clusters and would not be able to order the matrix in this way.) By doing this, we can explore similarity between observations by glimpsing "block structure" in the $n \times n$ matrix $X^\top X$, as below:

In [78]:
```
import matplotlib.pyplot as plt
import seaborn as sns

X_ND = data['X_ND']
_ = sns.heatmap(X_ND @ X_ND.T, cmap='Blues')
```



As you can see above, the block structure is very faint. We can instead use our model to get a clearer picture of the similarity between observations.

4a): Implement the affinity matrix.

In this question, you will run your EM algorithm on the data with the provided hyperparameters. Once converged, you will then use the results to compute the following **affinity matrix** $A \in (0,1)^{n \times n}$, where an entry A_{ij} contains the following posterior expectation:

$$A_{ij} = \mathbb{E}\left[z_i = z_j \mid X, \widehat{\Theta}^{\text{MAP}}\right] = \sum_{k=1}^{K} p(z_i = k, z_j = k \mid X, \widehat{\Theta}^{\text{MAP}})$$

Before running EM, **implement** `compute_affinity_matrix`.

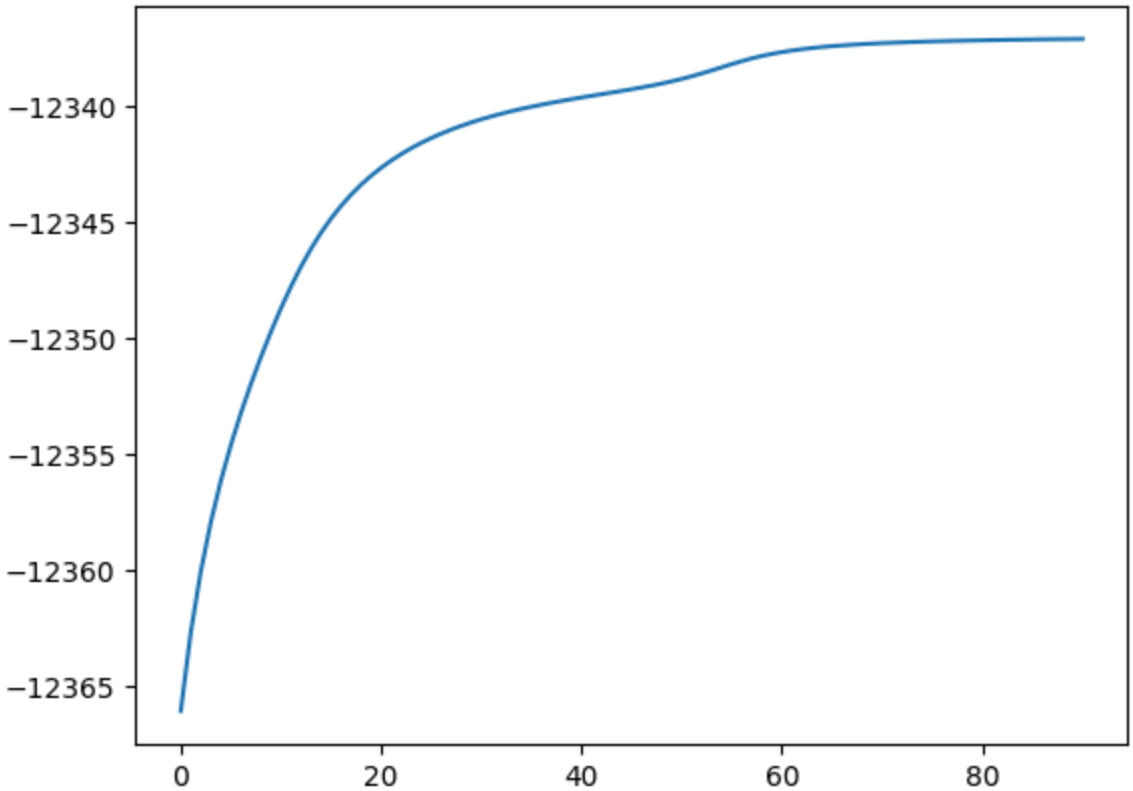
In [79]:
```
def compute_affinity_matrix(params_state, belief_state):
    """Compute the expectation of the similarity matrix given MAP estimates

    Args:
        params_state (dict): Dictionary containing the MAP estimates of the
        belief_state (dict): Dictionary containing the posterior expectatior

    NOTE: Both dictionaries are passed in by default, but you might not neec

    Returns:
        torch.Tensor: The computed similarity matrix A_NN of shape (n, n), w
                      the posterior expectation that observations i and j be
    """

    Ezeta_NK = belief_state['Ezeta_NK']

    A_NN = Ezeta_NK @ Ezeta_NK.T

    # A_NN = ...
    return A_NN
```

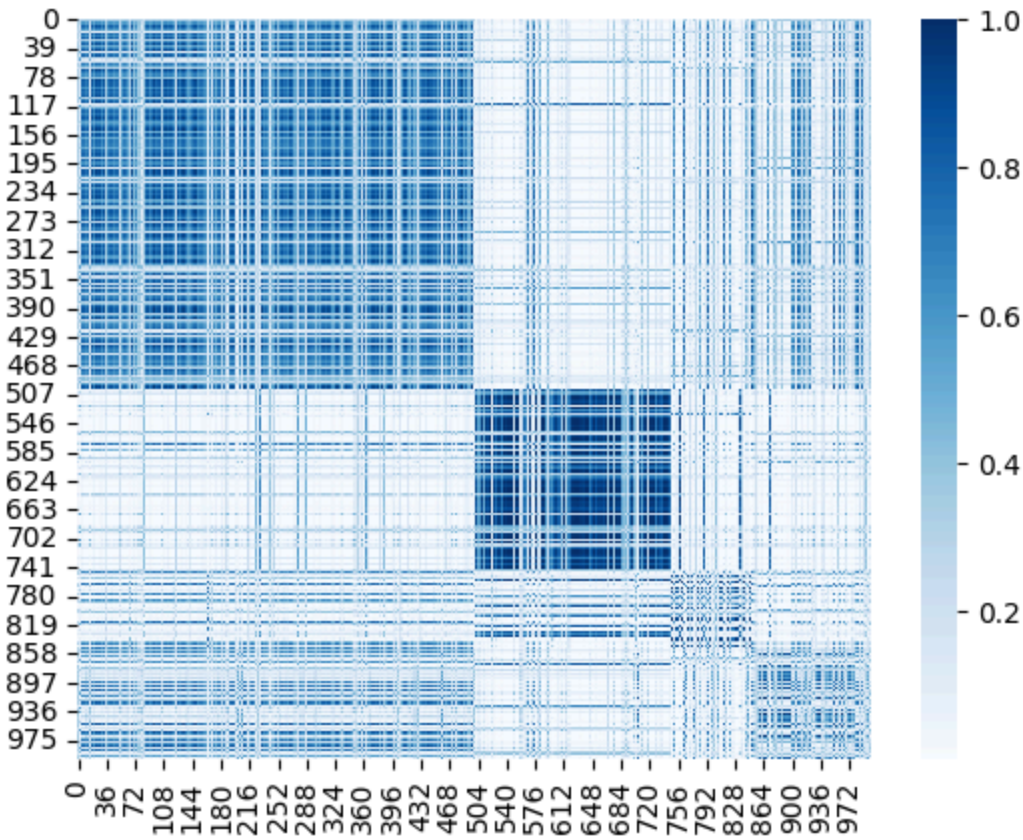
4b): Run EM on the data with the provided hyperparameters.

- Run the cell below.
- The output should display a **heatmap of A and a plot of the ELBO over iteration**.
- **Note:** The `em` function contains an assert statement which will crash if the ELBO ever decreases. **Do not comment this out.** If your ELBO is decreasing, there is a problem with your implementation, and you should return to previous steps and fix the issues.

In [80]:
```
# run EM
params_state, belief_state, elbo_vals = em(data, hyperparams, max_iter=100,

# compute affinity matrix
A_NN = compute_affinity_matrix(params_state, belief_state)

# plot the ELBO and the affinity matrix
plt.plot(elbo_vals[10:]); plt.show()
sns.heatmap(A_NN, cmap='Blues'); plt.show()
```

```
0%|          | 0/100 [00:00<?, ?it/s]
```



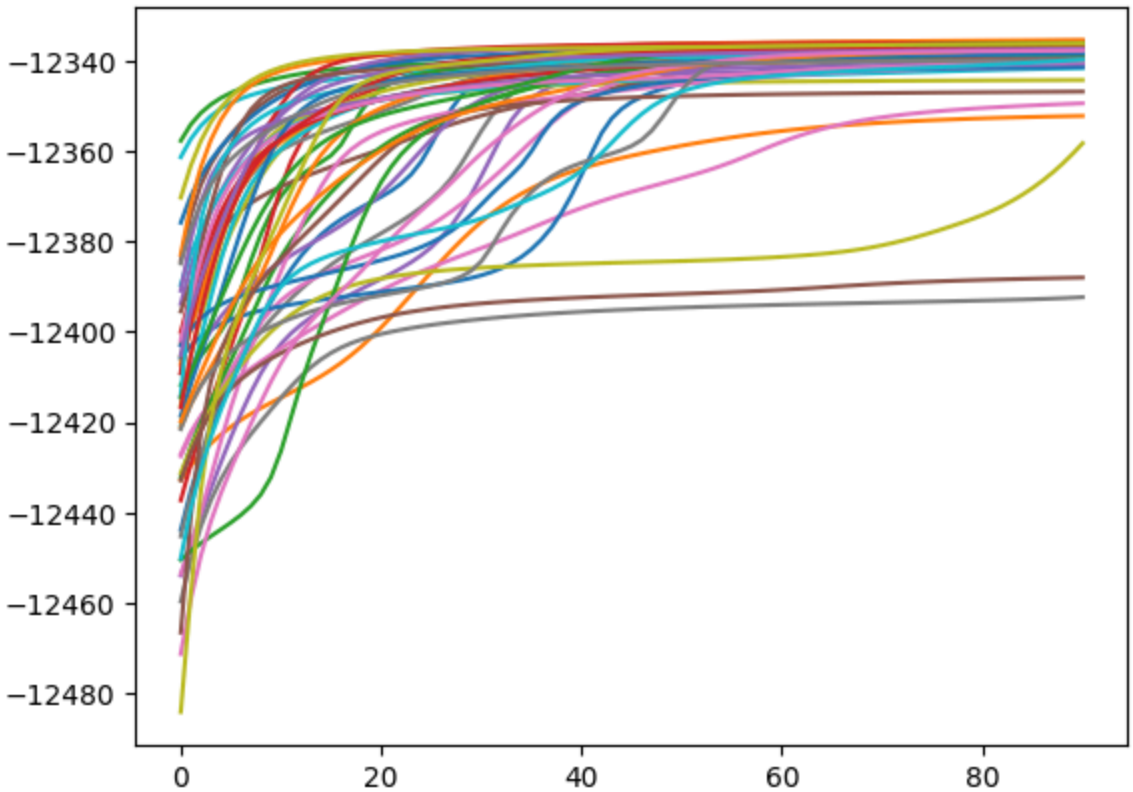


4c): Run EM with random restarts.

- Run the cell below.
- This will run EM fifty times with random restarts.
- Our implementation does this in under a minute. If yours is taking much longer, you should look at the bottlenecks in your code.
- The output should display 50 plots of the ELBO for the 50 random restarts.
- **Hint:** Our implementation returns ELBO values in the *rough* range of -12300.

In [81]:
```
best_params_state = {}
best_belief_state = {}
best_elbo_vals = [-torch.inf]
n_random_restarts = 50
for _ in tqdm(range(n_random_restarts)):
    seed = torch.randint(0, 10000, (1,)).item()
    params_state, belief_state, elbo_vals = em(data, hyperparams, max_iter=1
    plt.plot(elbo_vals[10:])
    if elbo_vals[-1] > best_elbo_vals[-1]:
        best_belief_state = belief_state
        best_params_state = params_state
        best_elbo_vals = elbo_vals

plt.show()
```

```
0%|          | 0/50 [00:00<?, ?it/s]
```



Question 5: Matrix completion [Code, results]

In this final problem, you will use your estimate $\widehat{\Theta}^{\text{MAP}}$ to do matrix completion on a test matrix Y.

The test matrix Y is $m \times D$ where the m rows correspond to $i = n, n+1, \ldots, n+m$ new observations.

The first three columns of Y are missing. The full matrix is $Y = [Y^{(\text{obs})}, Y^{(\text{miss})}]$ where $Y^{(\text{obs})}$ is $m \times (D-3)$ and $Y^{(\text{miss})}$ is $m \times 3$.

You will only observe $Y^{(\text{obs})}$. Your job is to use the MAP estimate of the parameters that you obtained in the previous problem (by fitting to X) to calculate the following posterior predictive expectation:

$$\mathbb{E}[Y^{(\text{miss})} \mid Y^{(\text{obs})}, X, \widehat{\Theta}^{\text{MAP}}]$$

- Your solution should not call `m_step` or any other function that updates the parameters.
- **Hint:** Your solution should make use of the `e_step` function you previously implemented. You do not need to change that function at all if you are clever about

what input you pass to it.

- **Hint:** We have created for you `params_state_obs`, a dictionary that contains portions of $\widehat{\Theta}^{\text{MAP}}$ which may be useful.

- Your code should ultimately create a tensor called `EY_MD_miss`, which contains the predictions of $Y^{(\text{miss})}$.

- After implementing your solution, run the cell so that it prints the RMSE on the missing portion of the test matrix and compares it to the RMSE of a naive estimator (which is based only on the empirical mean of $Y^{(\text{obs})}$).

- **Hint:** Our implementation returns test RMSE values less than the naive estimator.

In [82]:
```
with open('test_data.pkl', 'rb') as f:
    test_data = pickle.load(f)

Y_MD_obs = test_data['Y_MD_obs']

K = hyperparams['alpha_K'].shape[0]
m, D1 = Y_MD_obs.shape

params_state_obs = {'omega_KD': best_params_state['omega_KD'][:, 3:],
                    'lambd_KD': best_params_state['lambd_KD'][:, 3:],
                    'pi_K': best_params_state['pi_K']}



# 1. get cluster assignments
data_obs = {'X_ND': Y_MD_obs}
belief_state_test = {
    'Ezeta_NK': torch.ones((m, K)) / K,
    'Eb_NKD': torch.ones((m, K, D1)) * 0.75
}
belief_state_test = e_step(data_obs, belief_state_test,
                           params_state_obs, hyperparams)
Ezeta_MK = belief_state_test['Ezeta_NK']

# 2. missing columns
omega_KD_miss = best_params_state['omega_KD'][:, :3]
lambd_KD_miss = best_params_state['lambd_KD'][:, :3]

# E[y_{i,d} | z_i=k] = omega_{k,d} * lambda_{k,d}
E_y_given_k_KD = omega_KD_miss * lambd_KD_miss

# = sum_k q(z_i=k) * E[y_{i,d} | z_i=k]
EY_MD_miss = Ezeta_MK @ E_y_given_k_KD



rmse_model = torch.sqrt(torch.mean((EY_MD_miss - test_data['Y_MD_miss'])**2)
rmse_naive = torch.sqrt(torch.mean((Y_MD_obs.mean() - test_data['Y_MD_miss']
```

```
print(f'RMSE of model-based estimator: {rmse_model:.3f}')
print(f'RMSE of naive estimator: {rmse_naive:.3f}')
```

```
RMSE of model-based estimator: 1.203
RMSE of naive estimator: 1.329
```

In [ ]:

```
# TEST CELL

print("EY_MD_miss shape:", EY_MD_miss.shape, "(expected: (250, 3))")
print("EY_MD_miss >= 0?", (EY_MD_miss >= 0).all().item())
print("No NaN/Inf?", torch.isfinite(EY_MD_miss).all().item())
print(f"Model RMSE: {rmse_model:.3f}")
print(f"Naive RMSE: {rmse_naive:.3f}")
print(f"Model better than naive? {rmse_model < rmse_naive}")
```

```
EY_MD_miss shape: torch.Size([250, 3]) (expected: (250, 3))
EY_MD_miss >= 0? True
No NaN/Inf? True
Model RMSE: 1.203
Naive RMSE: 1.329
Model better than naive? True
```

Submission Instructions

Formatting: check that your code does not exceed 80 characters in line width. You can set *Tools* → *Settings* → *Editor* → *Vertical ruler column* to 80 to see when you've exceeded the limit.

Download your notebook in .ipynb format and use the following commands to convert it to PDF. First, rename your file `<yourlastname>_hw3.ipynb`. Then run:

```
jupyter nbconvert --to webpdf <yourlastname>_hw2.ipynb
```

This uses a headless browser to render the notebook as a PDF (no LaTeX installation required). Math is rendered via MathJax, so it should look the same as in the notebook.

Installing dependencies:

```
pip install "nbconvert[webpdf]"
playwright install chromium
```

Getting this HW into PDF form isn't meant to be a burden. If the above doesn't work, one quick and easy approach is to open it as a Jupyter notebook, print, save to pdf. Just make sure your latex math answers aren't cut off so we can grade them.

Please post on Ed or come to OH if there are any other problems submitting the HW.

Upload your .pdf file to Gradescope. Please tag your questions!